Software Testing and Quality Assurance

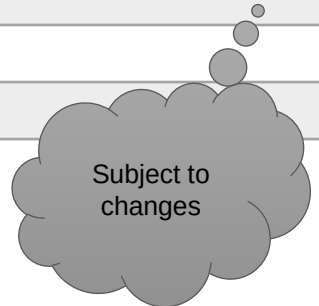
Lecture 3

Fabian Fagerholm

fabian.fagerholm@aalto.fi

Aalto University
School of Science

Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: White-box testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3



Quality characterisation model

(based on standards, tailored to the organisation)

Functionality | Reliability | Usability

Efficiency | Maintainability | Portability

Decomposition

Metrics

Selection, tailoring

Project-specific quality model

Project-specific quality requirements and attributes

Assure

Project-specific metrics and thresholds

Selection, tailoring

Abstract understanding of quality: conformance to requirements

High-level require-ments | High-level require-ments | High-level require-ments

Satisfy

Detailed specifi-cations | Detailed specifi-cations | Detailed specifi-cations | Detailed specifi-cations

Software Quality Management (SQM)

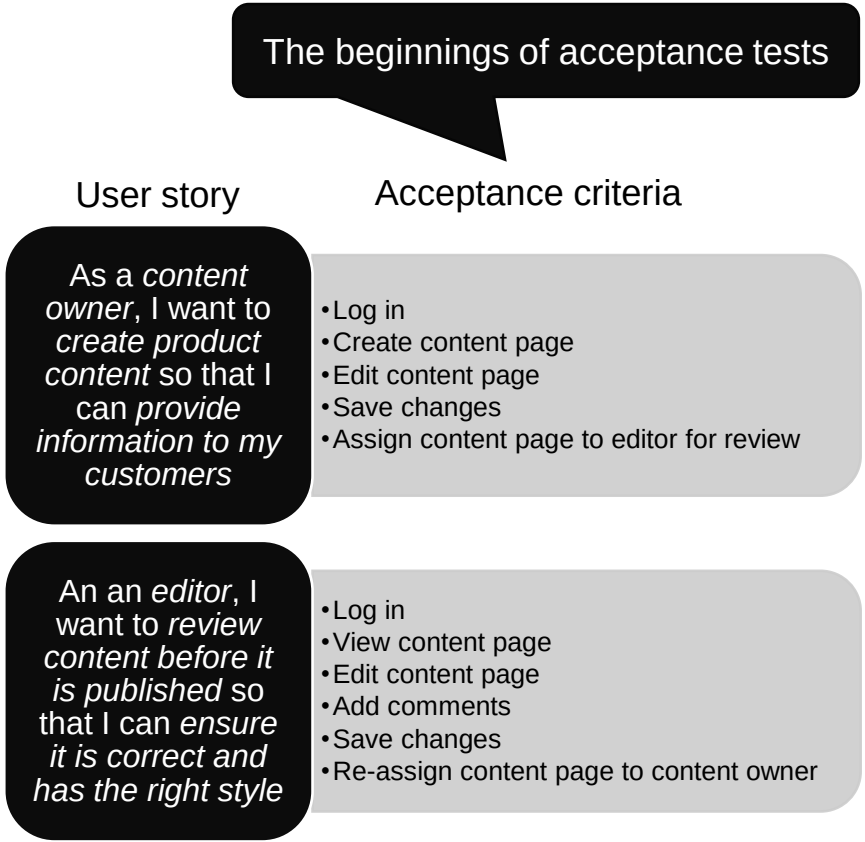
Software Quality Assurance (SQA)	Software Quality Control (SQC)	Software Quality Planning (SQP)	Software Process Improvement (SPI)
• **A general quality guide, not specific to a project**	• **Examine artefacts for compliance** • E.g. inspection, reviews, testing	• **Project-level quality commitment** • **Based on SQA**	• Improve process quality

Examine for compliance

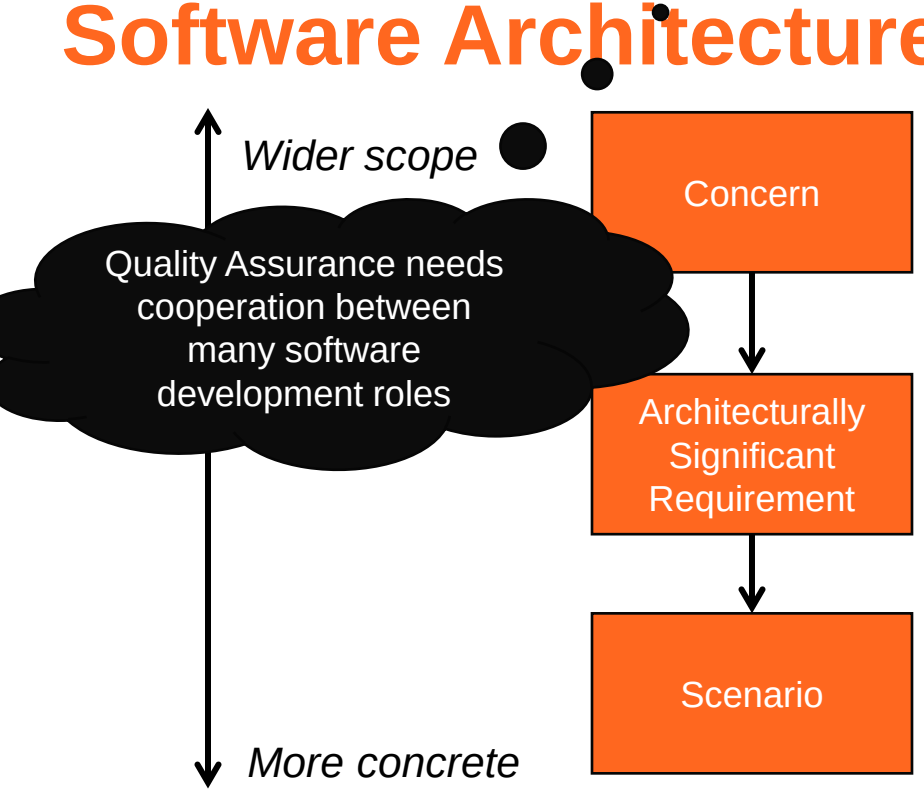
Aalto University
School of Science

User stories

- Functional descriptions written from a user perspective
- As a <*role*>, I can/want <*capability*> so that <*receive benefit*>
- Can be based on personas (descriptions of persons that represent real-life groups of users)
- User stories do not capture every kind of quality attribute – they most often describe only functionality



Linking Quality with Requirements and Software Architecture



Concern

Architecturally Significant Requirement

Scenario

The scenario can yield a quality requirement

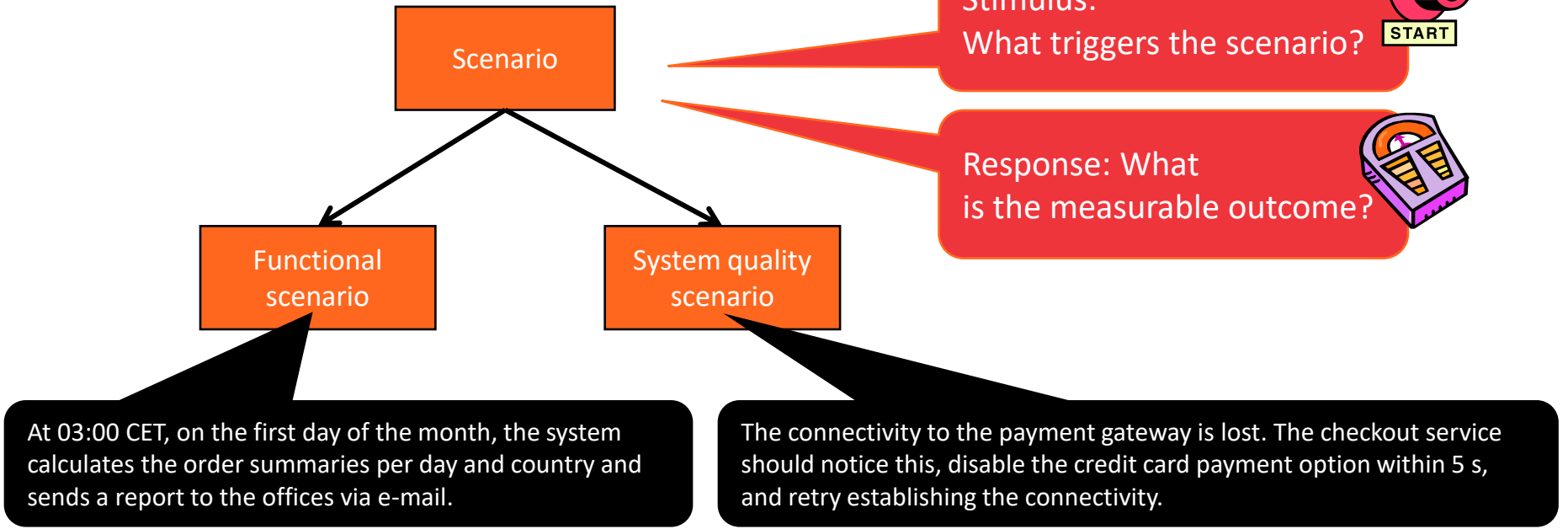
”The system should be available.”

Quality Attribute: Availability (depends on Reliability)

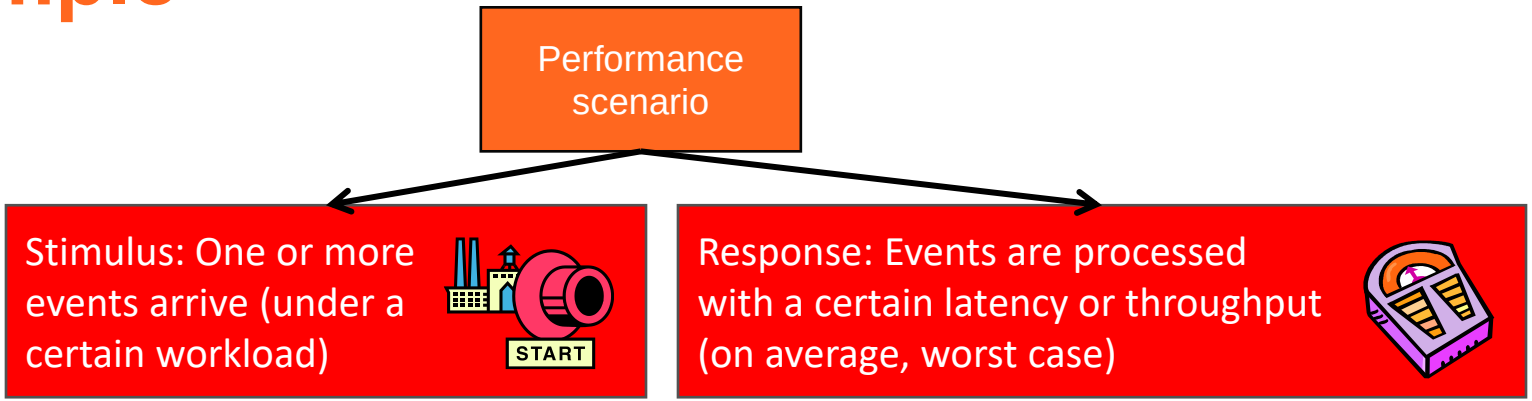
”The checkout service should have availability of 99,9%.”

”The connectivity to the payment gateway is lost. The checkout service should notice this, disable the credit card payment option within 5 s, and retry establishing the connectivity.”

Scenarios concretise requirements



Example



Latency: When the researcher presses *Start*, the simulation world with 100 creatures is initialized and animation begins within an average response time of one second.

Throughput: With 100 interacting creatures, the system can process, store and visualize at least 100 simulation steps per minute.

The beginnings of a performance test case

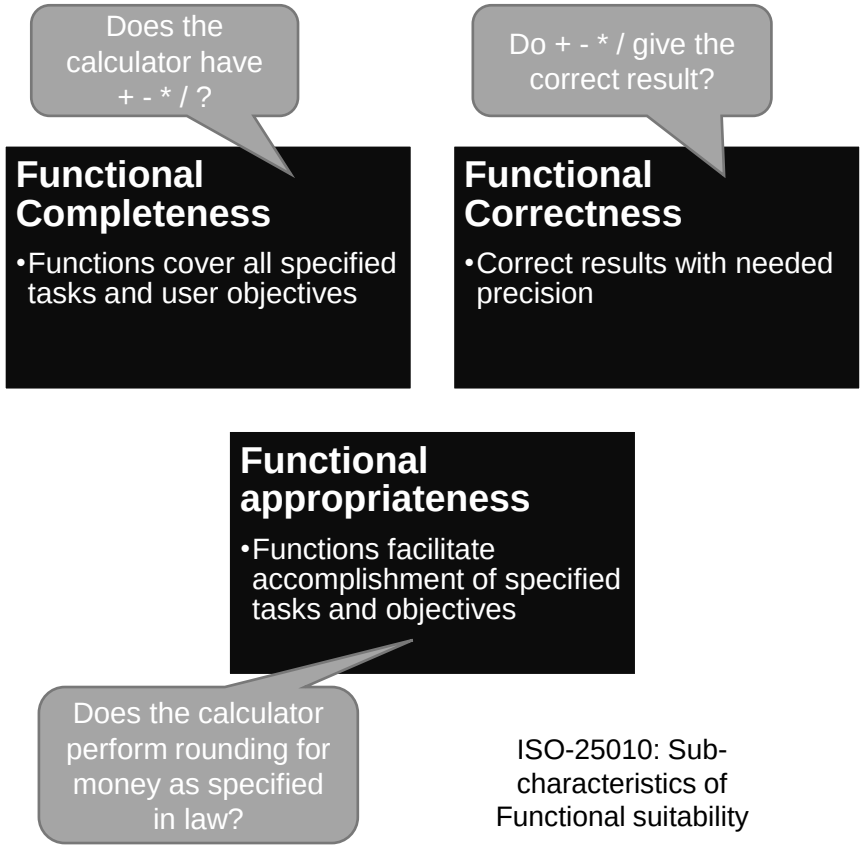
Software test case analysis and design

Aalto University
School of Science

Functional suitability

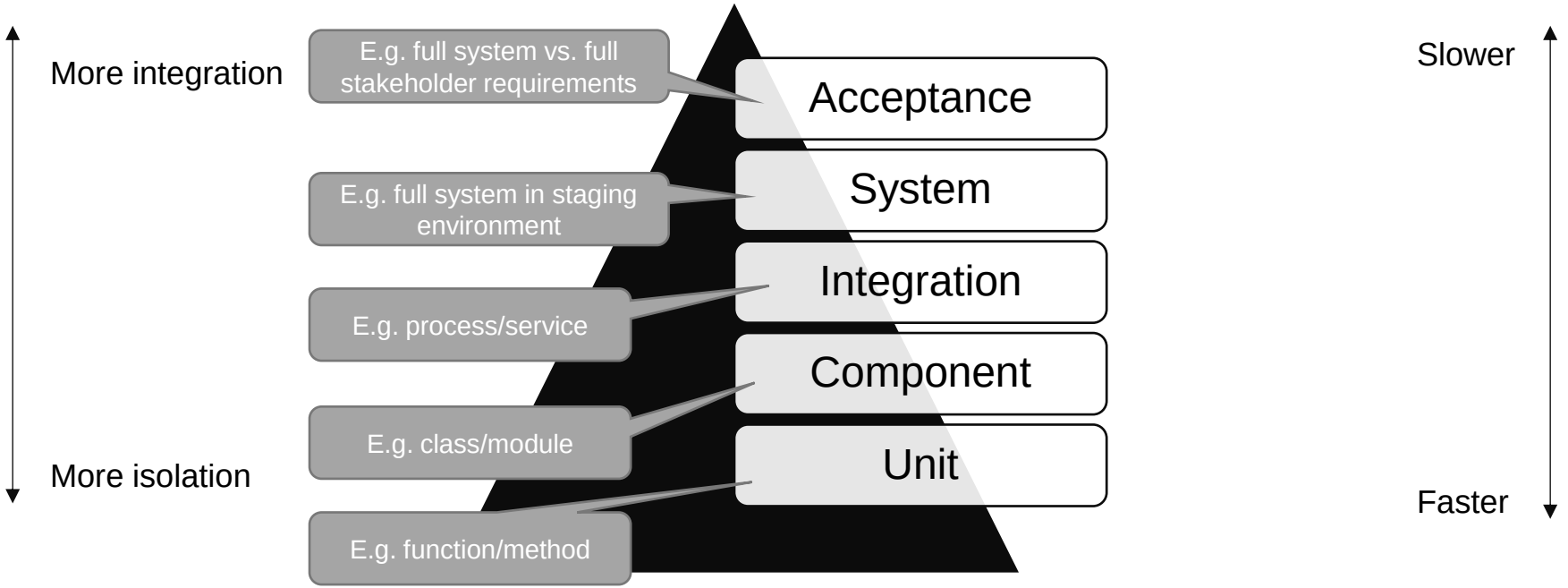
- The essential purpose of the software system
  - Simple: a basic calculator
  - More complicated: an internet banking system
- "Degree to which [system] provides functions that meet stated and implied needs when used under specified conditions" (ISO 25010)
  - Each individual function can be verified as existing or not existing
  - Specific properties can be specified for a function

Functionality is "just the beginning" of quality

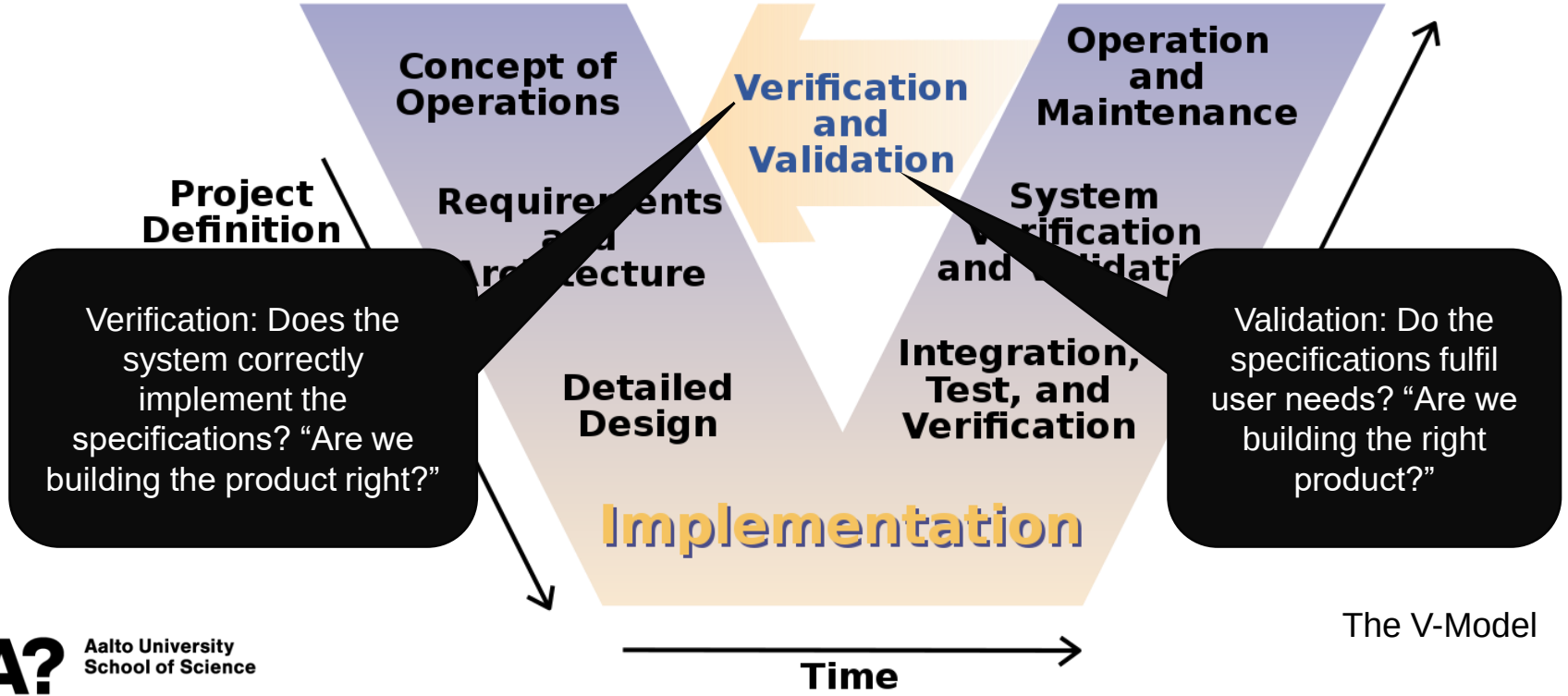


ISO-25010: Sub-characteristics of Functional suitability

Testing pyramid: levels of testing



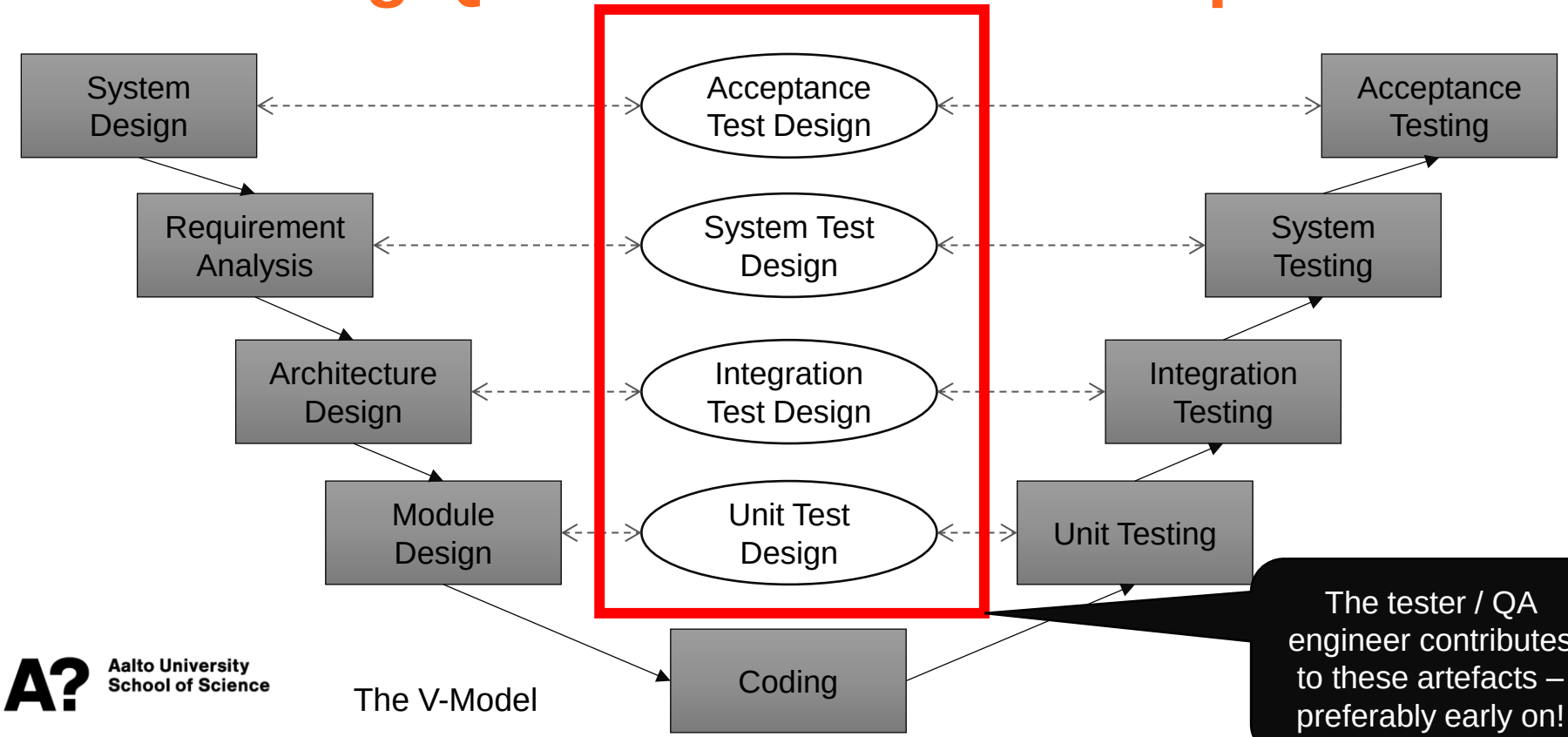
Positioning QA in software development



The V-Model

Aalto University
School of Science

Positioning QA in software development



The V-Model

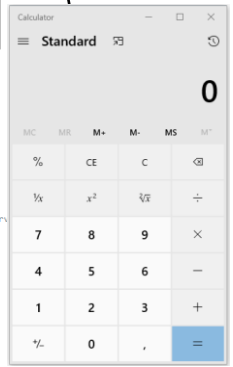
Different ideas of what testing is

Level 0	Level 1	Level 2	Level 3	Level 4
• No difference between testing and debugging. • No purpose of testing • Defects may be stumbled upon but no formalised effort to find them	• “The purpose of testing is to show that software works.” • Tests are designed to align with what the software does • Confirmation bias may play a role	• “The purpose of testing is to show that software doesn’t work.” • Challenges testers to find defects. • Consciously select test cases that evaluate the system under unusual circumstances, using “diabolical” test cases.	• “The purpose of testing is not to prove anything, but to reduce the perceived risk of not working to an acceptable value.” • Gain information about the quality of the software in terms of its defects • Provide management with an evaluation of the impact on our organisation if we ship the system in its present state	• “Testing is not an act. It is a mental discipline that results in low-risk software without much testing effort.” • Focus on making software testable from the start: reviews and inspections of its requirements, design, and code. • Write code with facilities that allows the tester to easily interrogate it while it is running. • Write self-diagnosing code that reports errors rather than requiring testers to discover them.

Beizer, B. (1990). Software Testing Techniques.

Concepts

System Under Test (SUT): The software system being tested

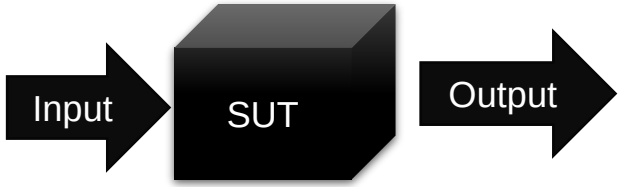



```
// Copyright (c) Microsoft Corporation. All rights reserv
// Licensed under the MIT License.

using CalculatorApp.ViewModel.Common;
using System;

namespace CalculatorApp
{
    namespace Converters
    {
        /// <summary>
        /// Value converter that translates true to false and vice versa.
        /// </summary>
        [Windows.Foundation.Metadata.WebHostHidden]
        public sealed class RadixToStringConverter : Windows.UI.Xaml.Data.IValueConverter
        {
            public object Convert(object value, Type targetType, object parameter, string language)
            {
```

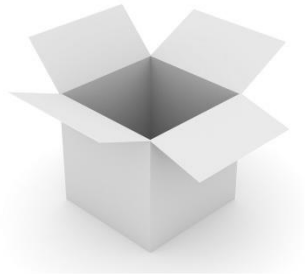
Aalto University
School of Science



Black-box testing: Observing SUT outputs for specified inputs *without looking at the internals*

Coverage: How much of the source code has been tested?

Grey-box testing: Black-box + White-box



White-box testing: Test SUT behavior *using knowledge of its internal structure*
(Also: clear-box, glass-box, or *structural testing)*

Discussion

Give an example of a useful software test.

What makes it useful?

Test case design

- A single test case can prove the system incorrect, but it is impossible to ever prove it correct
- You would have to test all valid and invalid combinations of input data
- A good test
  - Reveals faults in the current software
  - Reveals faults in the software when it is changed in the future (regression testing)
  - Does so efficiently and effectively – strikes a balance between how much time it takes to run the test and how much coverage is achieved
- To achieve this, test cases must be *designed*, not put together as an afterthought

Test case design

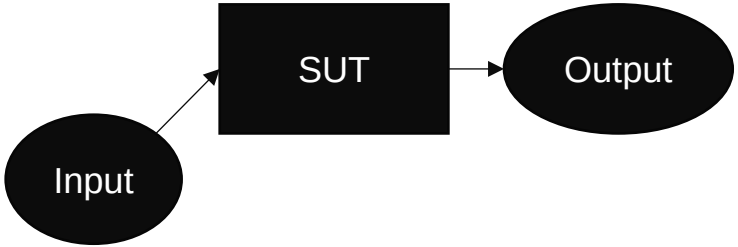
- Three parts of well-designed test cases

Inputs	Outputs	Order of execution

• Data entered by the user (keyboard, mouse, pointing gestures)
• Data from interfacing systems and devices
• Data from files and databases
• State of the system when data arrives
• Execution environment
• …
	• Data displayed on screen
	• Data sent to interfacing systems and devices
	• Data written to files and databases
	• Modified system state
	• Modified execution environment
	• …
		• *Cascading* test cases: each test case builds on the other
		  • Can be smaller, simpler
		  • If one case fails, the others may be invalid
		• *Independent* test cases: test cases do not depend on or influence each other
		  • Easier to isolate cause of failure
		  • Can be larger, more complicated

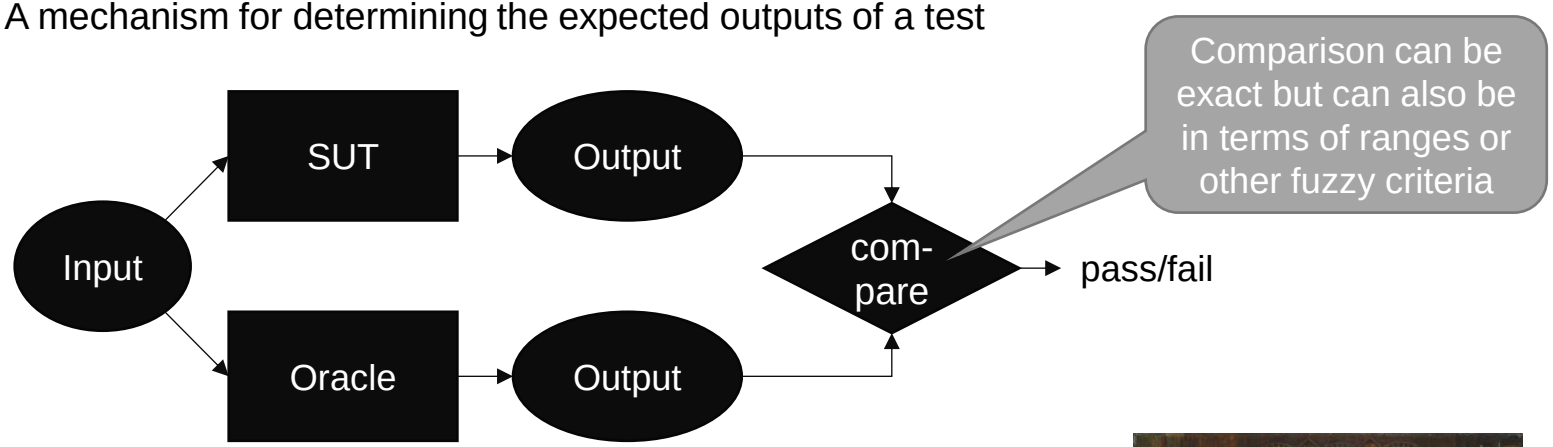
Correctness of Tests

- How do you know whether your test output is correct?



Test Oracle

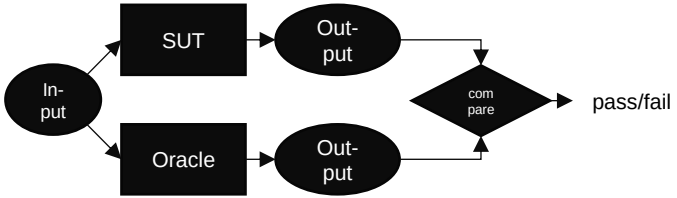
- Test Oracle: A mechanism for determining the expected outputs of a test



- But: authoritative (always correct) oracles might not exist (for all types of tests)
- Test Oracle (2): A tool that **helps decide** whether the program passed the test
- → Partial oracles (Weyker, 1980: On Testing Nontestable Programs)
- Where could we find partial oracles for different situations?

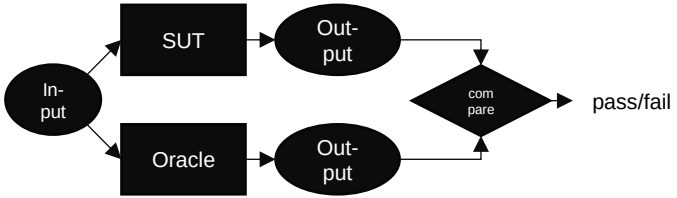
John William Waterhouse: Consulting the Oracle (1884). Public domain.

Test Oracle



- **Specified** oracles
  - Based on formal specifications
  - Model-based design: generate test oracles from formal models
  - Design by contract: assertions
- **Derived** oracles
  - Based on information derived from system artefacts (e.g. documentation, system execution results)
  - Regression testing: assumes that results from a previous system version can be used as an oracle for a future version
  - Performance testing: previous performance measures can be used as an oracle for future versions
  - Pseudo-oracle: a separately written program that takes the same inputs; compare outputs

Test Oracle

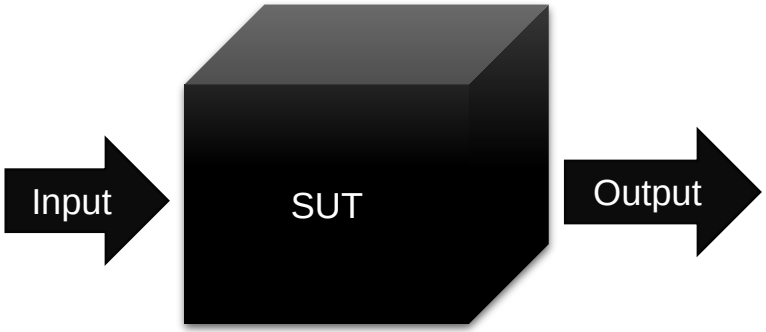


- **Implicit** oracles
  - Based on implied information and assumptions
  - Program crashes may imply a conclusion about unwanted behaviour (but a crash is not always a priority if the system can recover)
  - Negative testing, fuzz testing
- **Human** oracles
  - When other oracles cannot be used
  - Quantitative approach: find enough information about the SUT (e.g. test results) for a stakeholder to make decisions (e.g. release decision)
  - Qualitative approach: analyse SUT and its context to find representative and suitable inputs and outputs (e.g. using realistic input data and making sense of results)
  - Can be guided by heuristic approaches – gut instinct, rule of thumb, checklists, experience

Black-box testing

Black-box testing

- Testing that does not examine the internals of the SUT
  - Inputs
  - Outputs
- Also called *functional testing, behavioural testing*, or *specification-based testing*
- Can be applied on all levels of testing (unit, integration, system, acceptance)
- Automated tests can be black-box tests
- Exploratory testing is a kind of black-box testing
- Advantage: can be efficient and effective
- Disadvantage: can never be sure of how much of the system has been exercised
- We focus here on general techniques that can be applied in more rigid tests (both manual and automated)



Some very specific inputs could lead to a rare execution path that cannot be known without seeing the code

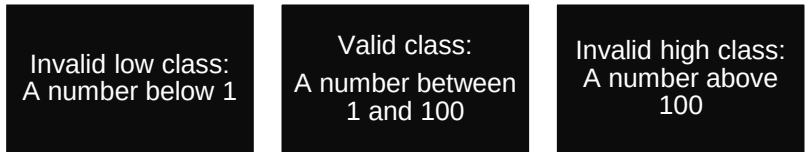
```
if (student.number == 1234567) {
    student.grade = 5;
}
```

Equivalence partitioning

- Divide test input data into a set of classes
- Select one input value from each class
- The input value represents the class
- Create one test with each chosen input value
- → One test per class
- Significantly reduces number of test cases
- Keeps reasonable test coverage

Example

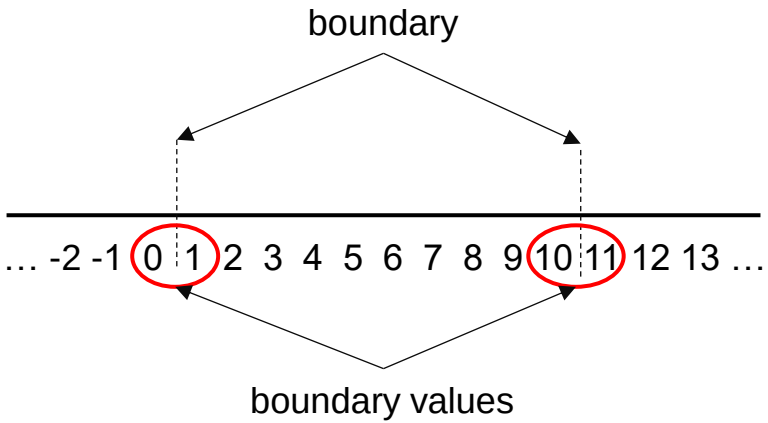
- Inputs: integers 1-100
- 100 tests?
- With equivalence partitioning:



Boundary value analysis (BVA)

- Assumption: defect density is clustered towards value range boundaries
  - Errors in implementing comparisons (< vs. <=)
  - Errors in implementing loop termination conditions
  - Errors in interpreting requirements at the boundaries
- Test cases can concentrate on these boundary values
- Identify equivalence classes
- Identify boundaries of each equivalence class
- Create test cases for:
  - One point just above the boundary
  - One point just below the boundary

Example:
Valid inputs: integers 1-10

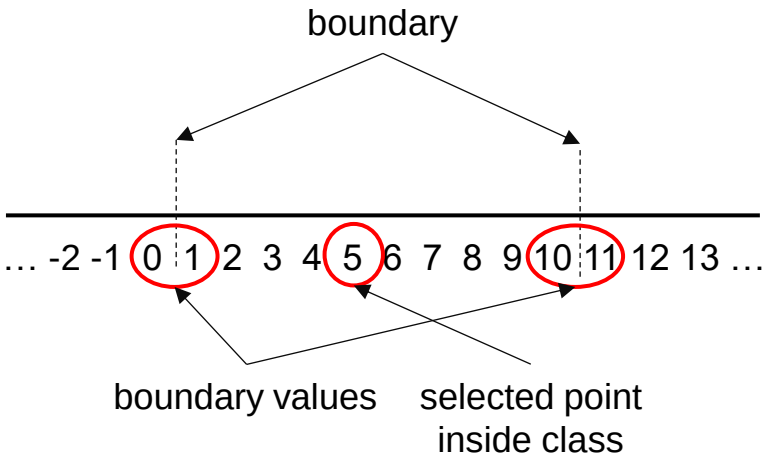


Robustness testing (variant of BVA)...

- Testing across the valid and invalid partitions
- Create test cases for:
  - One point just above the boundary
  - One point just below the boundary
  - One point inside the class

- BVA and its variants can be used to generate systematic tests and reduce the number of test cases
- Multiple parameters: only change one parameter at a time

Example:
Valid inputs: integers 1-10

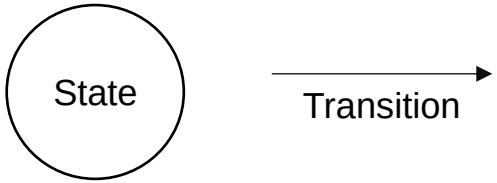


Decision table testing

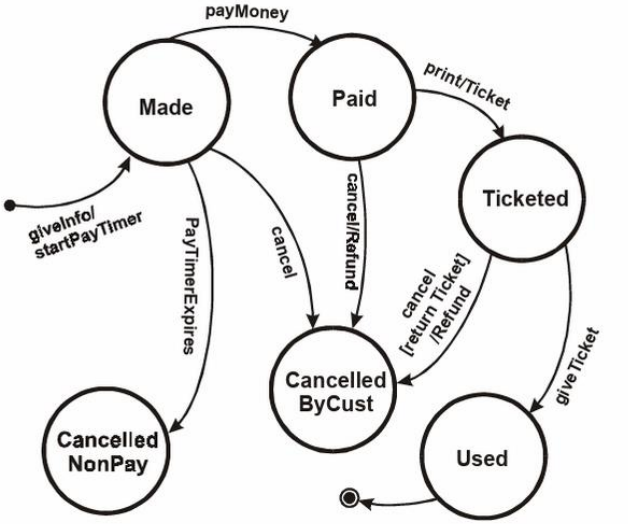
- Decision table:
  - A systematic way of recording complex business rules
  - Serves as a guide to creating test cases
- List all *input conditions* that can occur and all *actions* that can arise from them
  - Input conditions on the top
  - Resulting actions on the bottom
  - Each column represents a business *rule*
  - Each column becomes a test case
- If the conditions are value ranges, consider testing at both the low and high end of the range (compare: BVA/robustness testing)
- Every possible combination does not need to be entered, only those that represent actual business rules

	Rule 1	Rule 2	Rule 3	Rule 4
Input conditions				
Customer with loyalty card?	Yes	Yes	No	No
Spend > 200?	Yes	No	Yes	No
Actions				
Discount	10%	5%	2%	0%
Accumulate loyalty points?	Yes	Yes	No	No
	Test case 1	Test case 2	Test case 3	Test case 4

State transition testing

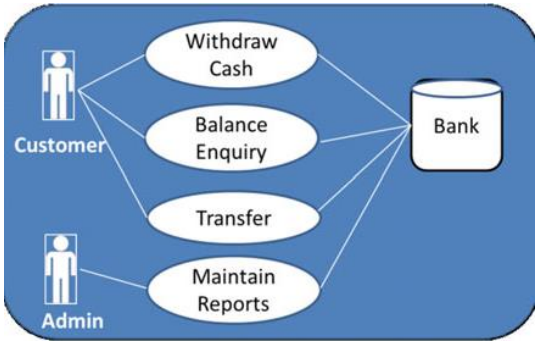


- Sometimes outputs depend on system state: system behaviour is triggered by state transitions
- State: a “stable” system condition – does not change unless a trigger event occurs
- Transition: a description of the trigger that moves the system from one state to another
- Create a state diagram of the system’s states and transitions
- Can also create a state table
- Test cases can be created depending on desired coverage
  - All states visited at least once (weak)
  - All events triggered at least once (weak)
  - All paths executed at least once (stronger)
  - All transitions exercised at least once (strongest)

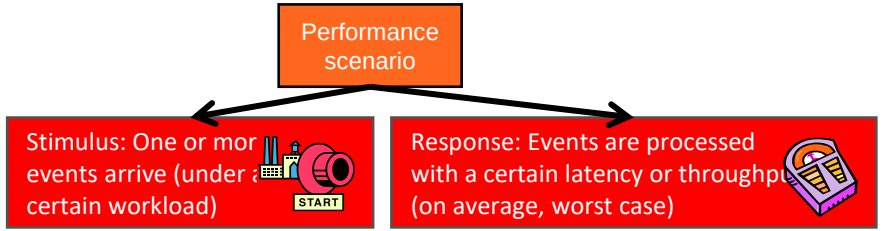


Copeland. (2003). A Practitioner’s Guide to Software Test Design.

Use case and scenario-based testing



- Use cases often lack necessary detail for testing
- Add details, e.g.
  - Preconditions
  - Main success scenario
  - Success end conditions
  - Failed end conditions
  - Response time
  - Frequency
  - …



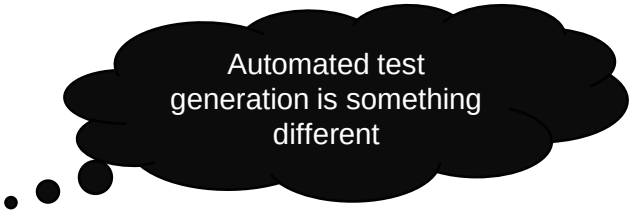
Latency: When the researcher presses *Start*, the simulation world with 100 creatures is initialized and animation begins within an average response time of one second.

Throughput: With 100 interacting creatures, the system can process, store and visualize at least 100 simulation steps per minute.

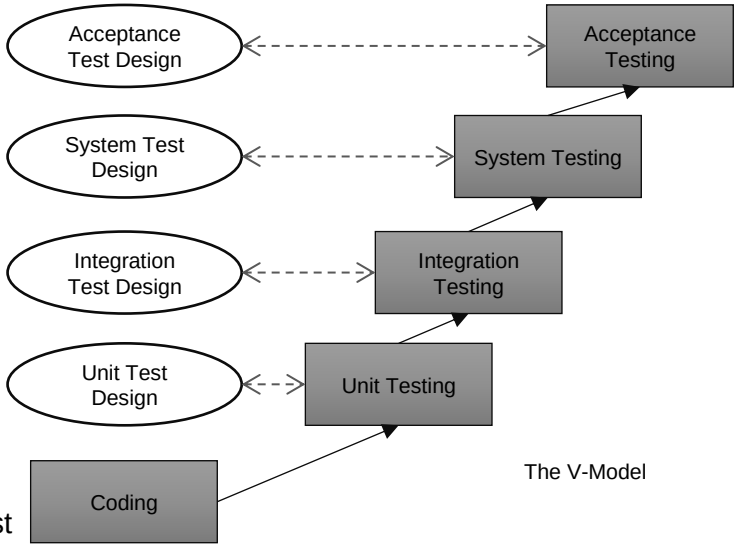
The beginnings of a performance test case

A detailed specification is an important input to test design

Test automation



- Automated tests use software programs that take another program as input and produce a test result as output
- The test can be performed on the source code (static) or the running program (dynamic)
- Usually a test framework and a test runner are used
- The program to test is called the **System Under Test** (SUT)
- Types of tests (examples)
  - Unit tests
  - Acceptance tests
  - Performance tests
  - Load tests
  - …
- Automated test *design* is often more time-consuming than manual test design because nothing can be left open to interpretation
- Automated test *execution* can be faster than manual – but running the whole test suite can be slow



The V-Model

Test automation: Unit test example

```
# System Under Test


class StudentDirectory:
    names = []


    def add_name(self, name):
        if name not in self.names:
            self.names.append(name)
```

```
# Test that names are only added once


directory = StudentDirectory()
directory.add_name("Emma")
directory.add_name("Emma")

if len(directory.names) != 1:
                print("Test failed")
else:
                print("Test passed")
```

Some important guidelines for unit tests

- **Well named**: The test name explains what it tests
- **Clearly defined**
  - Each test tests one thing
  - Inputs and outputs are unambiguous
- **Repeatable (deterministic)**: The test gives the same results no matter how many times it is run
- **Isolated**
  - No interactions or dependencies between tests
  - No dependencies on outside factors (e.g., file system, database)
  - Each test leaves the system in a well-defined state
- **Robust**: The test does not break if the unit internals are changed (related to isolation)
- **Fast**: The total testing time is reasonable even with thousands of tests
- **Documented**: Explain the purpose of the test, the test scenario, and the expected behaviour when the scenario is invoked

Next steps

Use the related reading for more details. Reserve enough time to set up your environment.

- Individual assignments in MyCourses
  - Quiz on equivalence partitioning
  - Quiz on boundary value analysis
  - Unit testing 1

Deadline: before next lecture (Tuesday by 10:00)

- Group assignments in MyCourses
  - Group registration, Group name choice (DL: 20.9 by 16:00)
  - Analysing quality using ISO/IEC 25010 (DL: 24.9)
- Getting help
  - Ask in the course chat, see Communication section in MyCourses
  - Personal questions by email

Study smarter, not harder!

- **Plan: when and where**
- **Go deep at your own pace**
- **Get some rest**
- **Practice makes perfect**
- **Enjoy it ☺**